

A Physicist's Guide to the 2D Ising Model Simulation: From Physical Principles to Python Code

A Guided Translation

October 31, 2025

Abstract

This document serves as a self-contained manual for understanding a Monte Carlo simulation of the two-dimensional Ising model. We adopt a physicist's perspective, focusing not on programming idioms but on the direct translation of physical concepts into computational instructions. We will deconstruct a canonical Python code snippet to reveal the underlying physics in each line, from the definition of the lattice and its interactions to the implementation of the Metropolis algorithm that simulates contact with a thermal reservoir.

1 The Physical Model

Before writing a single line of code, we must first define the physical universe we intend to simulate.

1.1 The Lattice and Spins

Our system is a two-dimensional square lattice of size $L \times L$. At each site, indexed by coordinates (i, j) , there exists a classical spin $s_{i,j}$ which can only take one of two values: $s_{i,j} \in \{+1, -1\}$, representing a microscopic magnetic moment pointing "up" or "down". A complete specification of all spin values on the lattice, $\mathbf{s} = \{s_{i,j}\}$, defines a single microstate of the system.

1.2 The Hamiltonian: The Law of Energy

The interactions between spins are governed by the Ising Hamiltonian, H . For a ferromagnetic system with only nearest-neighbor interactions and no external magnetic field ($h = 0$), the Hamiltonian is given by:

$$H(\mathbf{s}) = -J \sum_{\langle(i,j),(k,l)\rangle} s_{i,j} s_{k,l}$$

where:

- $J > 0$ is the ferromagnetic coupling constant, which sets the energy scale of the interaction. A positive J means that neighboring spins prefer to align.
- The notation $\langle(i,j),(k,l)\rangle$ denotes a sum over all unique pairs of nearest-neighbor sites on the lattice.

The negative sign ensures that aligned neighbors ($s_{i,j}s_{k,l} = +1$) contribute a lower energy ($-J$) to the system, which is the energetically preferred state. For simplicity in our simulation, we will work in units where $J = 1$ and the Boltzmann constant $k_B = 1$.

1.3 Boundary Conditions: Defining the Geometry

To simulate an infinite, or "bulk," material and avoid spurious edge effects, we impose **periodic boundary conditions (PBC)**. This means the lattice is treated as the surface of a torus. A spin at the rightmost edge of the grid ($i = L - 1$) considers the corresponding spin at the leftmost edge ($i = 0$) as its right-hand neighbor, and vice-versa. The same applies to the top and bottom edges.

$$s_{i,L} \equiv s_{i,0} \quad \text{and} \quad s_{L,j} \equiv s_{0,j}$$

2 The Simulation Algorithm: The Metropolis Method

Our goal is to simulate the system in contact with a heat bath at a constant temperature T . In thermal equilibrium, the probability of observing any microstate $\mathbf{s}$ follows the Boltzmann distribution:

$$\pi(\mathbf{s}) = \frac{1}{Z} e^{-\beta H(\mathbf{s})}, \quad \text{where} \quad \beta = \frac{1}{T}$$

The Metropolis-Hastings algorithm is a Markov Chain Monte Carlo (MCMC) method designed to generate a sequence of states that are samples from this very distribution.

2.1 The Local Update and Detailed Balance

The algorithm proceeds via a sequence of proposed local updates. The most common choice is a single-spin flip. The key physical quantity governing this process is the change in energy, ΔE , resulting from flipping a single spin at site (i, j) . Due to the nearest-neighbor nature of the Hamiltonian, this is a purely local calculation:

$$\Delta E = 2J s_{i,j} (s_{i-1,j} + s_{i+1,j} + s_{i,j-1} + s_{i,j+1})$$

The Metropolis algorithm then provides a rule for accepting or rejecting this proposed flip, which satisfies the detailed balance condition and guarantees convergence to the Boltzmann distribution:

1. Propose a flip by randomly selecting a spin $s_{i,j}$.
2. Calculate the energy change ΔE that this flip would cause.
3. Accept the flip with probability $P_{\text{accept}} = \min(1, e^{-\beta \Delta E})$.

This acceptance rule is the physical heart of the simulation. Energy-decreasing flips ($\Delta E \leq 0$) are always accepted. Energy-increasing flips are accepted with a probability that is exponentially suppressed, perfectly capturing the nature of thermal fluctuations.

3 Translating Physics into Code

We now translate the physical model and algorithm described above into the provided Python code.

3.1 Representing the System and Initial State

First, we define the lattice and initialize it in a state of maximum disorder (corresponding to $T = \infty$).

```
1 import numpy as np
2 import random
3 import math
4
5 # Setup
6 size = 50                                # lattice size
7 s = np.ones((size, size), dtype=int)    # spin array: +1 or -1
8
9 # Initialize with random spins
10 for i in range(size):
11     for j in range(size):
12         s[i,j] = 1 if random.random() < 0.5 else -1
```

Listing 1: System setup and random initialization.

Translation:

- `size = 50`: This sets the physical dimension, $L = 50$.
- `s = np.ones(...)`: This creates the $L \times L$ grid to hold the spin states $\{s_{i,j}\}$. A NumPy array is our computational representation of the physical lattice.
- The nested `for` loops implement a "hot start" by assigning $s_{i,j} = \pm 1$ with equal probability. This prepares the system in a state of maximum entropy, a physically unbiased starting point.

3.2 The Energy Calculation Engine

Next, we implement the formula for the local energy change ΔE , including the periodic boundary conditions.

```
1 # Core physics: Energy change calculation (periodic boundary conditions)
2 def deltaE(i, j):
3     """Energy change if spin at (i,j) flips"""
4     leftS = s[size-1, j] if i == 0 else s[i-1, j]
5     rightS = s[0, j] if i == size-1 else s[i+1, j]
6     topS = s[i, size-1] if j == 0 else s[i, j-1]
7     bottomS = s[i, 0] if j == size-1 else s[i, j+1]
8     return 2.0 * s[i, j] * (leftS + rightS + topS + bottomS)
```

Listing 2: Calculating the energy change of a proposed flip.

Translation:

- The `if...else` statements are the direct implementation of the periodic boundary conditions. For instance, `leftS = s[size-1, j] if i == 0 else s[i-1, j]` translates to: "The left neighbor of a spin at index $i = 0$ is the spin at index $i = L - 1$; otherwise, it is the spin at $i - 1$."
- The `return` statement is a line-for-line coding of our boxed equation for ΔE , with $J = 1$.

3.3 The Heart of the Simulation: The Metropolis Step

Finally, we encode the Metropolis acceptance rule itself.

```
1 # Metropolis algorithm - the heart of the simulation
2 def monte_carlo_step(temperature):
3     """Perform one Monte Carlo step"""
4     i = random.randint(0, size-1)
5     j = random.randint(0, size-1)
6     eDiff = deltaE(i, j)
7
8     # Accept flip if energy decreases OR with probability exp(-Delta_E/T)
9     if eDiff <= 0 or random.random() < math.exp(-eDiff/temperature):
10         s[i, j] = -s[i, j]
```

Listing 3: A single Monte Carlo update step.

Translation:

- `i = random.randint(...), j = random.randint(...)`: This is Step 1, proposing an update by selecting a random lattice site.
- `eDiff = deltaE(i, j)`: This is Step 2, evaluating the energy cost using our physics engine.
- The `if` statement is the elegant implementation of Step 3, the acceptance rule. The condition `eDiff <= 0` handles the deterministic acceptance of favorable flips. The condition `random.random() < math.exp(-eDiff/temperature)` translates the probabilistic acceptance of unfavorable flips, where we compare a random number to the Boltzmann factor $e^{-\beta\Delta E}$.
- `s[i, j] = -s[i, j]`: This line executes the flip if the acceptance condition is met, updating the state of our universe.

4 Running the Experiment

The final piece of code sets the experimental conditions and runs the simulation.

```
1 # Run simulation
2 temperature = 2.27 # critical temperature
3 for step in range(100000):
4     monte_carlo_step(temperature)
```

Listing 4: The main simulation loop.

Interpretation:

- `temperature = 2.27`: We set the temperature of the heat bath. The value $T \approx 2.269$ is the known critical temperature for the 2D Ising model. Simulating near this temperature allows us to observe the fascinating physics of phase transitions, such as critical slowing down and the formation of fractal-like domains.
- `for step in range(...)`: This is the clock of our simulation. It iterates the `monte_carlo_step` function, evolving the system in "Monte Carlo time." After a sufficient number of steps for the system to thermalize, the subsequent configurations are fair samples from the true equilibrium ensemble at the chosen temperature.